ROS 2 Web Teleoperation Study Guide

A practical learning path from simulation to reliable robot control

Prepared for Gajendra | IIT Patna | 5 November 2026
Reference status | Original upstream code pinned at 011692aeed3f | MIT license
Original repository

Purpose and status

This guide explains how to learn from RoboPilot, a small browser-controlled differential-drive simulation. The aim is to understand the full command and telemetry path, then design an original reliability and logging extension. The upstream project belongs to weepwood. Its code is included as a pinned reference submodule; the guide does not present it as Gajendra's work. The ROS stack was reviewed from source but was not run on this Windows machine.

Abstract

RoboPilot combines a ROS 2 Jazzy robot description, Gazebo Harmonic world, a ROS to Gazebo bridge, rosbridge and a Vue dashboard. The browser publishes velocity commands and displays odometry and lidar data. The learning sequence begins with reading the message graph, proceeds to a controlled simulation run, and ends with failure testing and a separate robot-side command timeout design. These skills transfer to robot bring-up, teleoperation, diagnostics and data collection work.

Why this reference is useful

The repository is focused enough to trace end to end. It uses the current documented ROS 2 Jazzy and Gazebo Harmonic pairing on Ubuntu 24.04. It exposes standard topics that appear in many mobile robots. Its browser controls also create concrete questions about stale commands, network loss and operator feedback. The project has only a small public footprint, so assess it by reading and testing the code rather than assuming it is production ready.

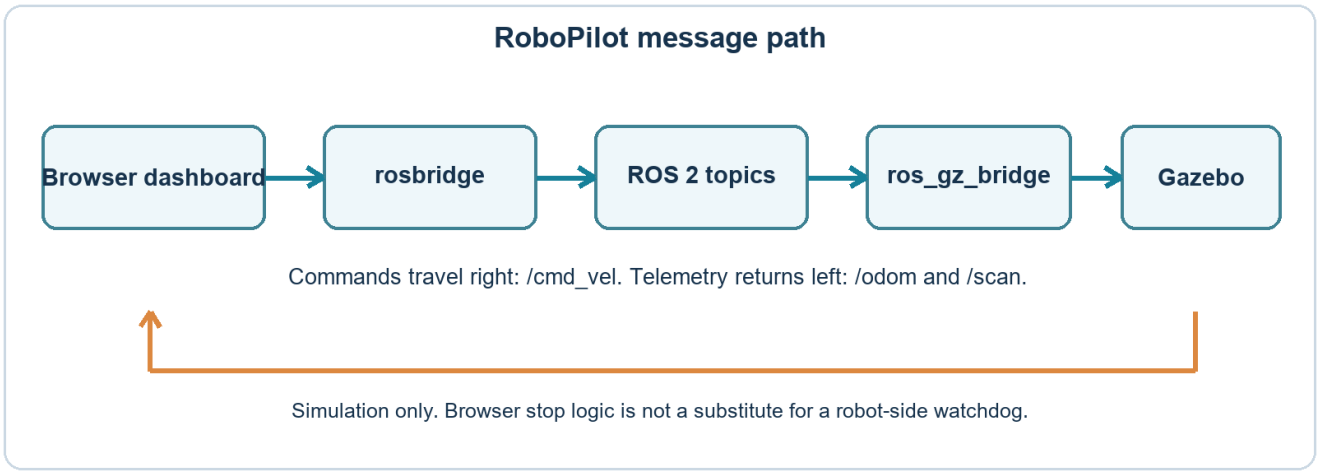


Figure 1 Command and telemetry path inferred from the pinned upstream files

Concepts to learn before running

Concept	Meaning in this project	Check in a run
ROS 2 topic	Named stream between publishers and subscribers	ros2 topic list and ros2 topic info
Twist command	Linear x and angular z velocity request on /cmd_vel	Echo commands while pressing and

Concept	Meaning in this project	Check in a run
		releasing controls
Odometry	Estimated position and velocity on /odom	Check sign and units while moving
LaserScan	Ranges around the robot on /scan	Inspect valid ranges and frame ID
TF	Transforms between robot coordinate frames	Inspect frame tree and timestamps
rosbridge	WebSocket bridge between browser and ROS graph	Confirm URL and connection state

For a differential drive robot, forward speed is half the sum of wheel linear speeds. Turning rate depends on their difference divided by wheel separation. You do not need to derive the full controller at first; you should understand why a Twist command can make the wheels turn at different rates.

Read the upstream files in this order

File	Question to answer
README.md and docs/ARCHITECTURE.md	What components start and which ports or topics connect them?
src/robopilot_sim/urdf/robopilot.urdf.xacro	Which links, joints and sensor frames make up the robot?
src/robopilot_sim/config/bridge.yaml	Which topics move from ROS to Gazebo and back?
src/robopilot_sim/launch/simulation.launch.py	Which processes start, and what happens when a launch option is disabled?
dashboard/src/App.vue	Where are commands published, stopped and displayed?
scripts/build_ros.sh and scripts/run_sim.sh	What environment do the helper scripts assume?

In App.vue, find startMotion, publishActiveMotion, stopMotion and emergencyStop. The browser publishes commands repeatedly while an input is active. The code calls stopMotion on key release, blur and visibility change. A lost connection can prevent the last zero command from reaching a robot. Treat the displayed E-STOP as a software state in this simulation, not as a physical safety circuit.

Environment and first run

Use Ubuntu 24.04 with ROS 2 Jazzy and Gazebo Harmonic, either natively or through Windows 11 WSL2 with WSLg. The upstream Windows guide recommends keeping a runnable workspace inside the Linux filesystem rather than under /mnt/d. Installations and package versions should follow the upstream README and official ROS documentation at run time.

```
# In an Ubuntu 24.04 shell after installing ROS 2 Jazzy
mkdir -p ~/projects && cd ~/projects
git clone https://github.com/weepwood/RoboPilot.git
cd RoboPilot
# Review scripts/bootstrap_ubuntu.sh before running it
./scripts/build_ros.sh
./scripts/run_sim.sh
# In a second shell: ./scripts/run_dashboard.sh
```

These commands are a planned procedure, not a completed run. The bootstrap script installs software, so read it and confirm the package list first. A successful baseline requires Gazebo and RViz to open, a browser dashboard to connect, and /cmd_vel, /odom and /scan to appear with expected message types.

Experiments and evidence

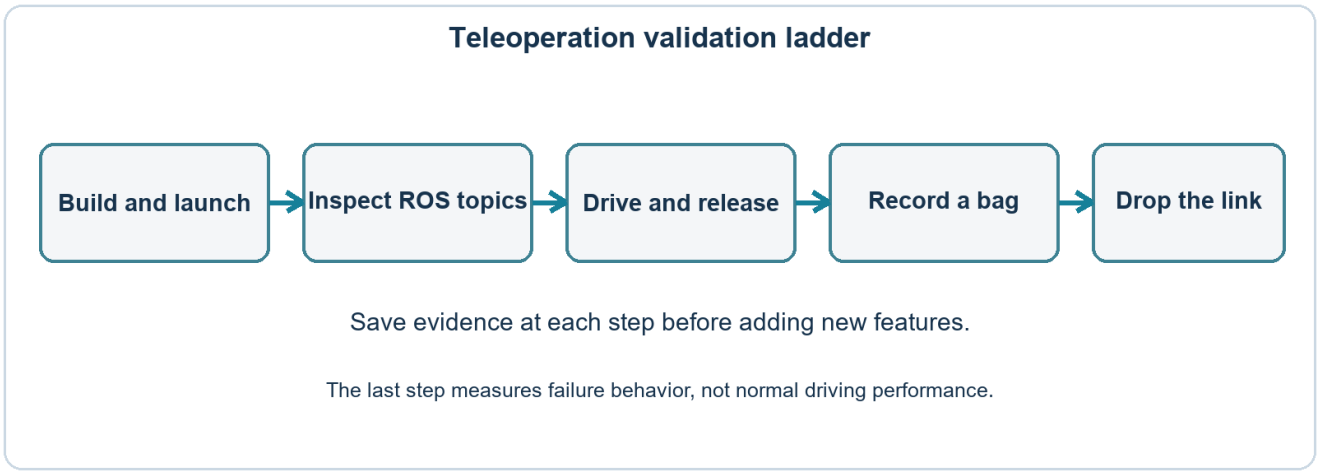


Figure 2 Study sequence with an evidence checkpoint after each stage

Experiment	Action	Evidence to save
Graph inventory	List nodes, topics, topic types and frame IDs	Command output with date and ROS version
Drive and release	Hold forward, turn, release and press Stop	Short /cmd_vel log and /odom plot
Input loss	Blur tab, close browser, interrupt WebSocket	Last command and time until motion stops
Record and replay	Record /cmd_vel /odom /scan with ros2 bag	Bag metadata and replay notes
Parameter change	Change velocity limits and compare	Before and after values, run conditions

Use ros2 bag record /cmd_vel /odom /scan for a short reproducible session and ros2 bag info on the saved directory. Do not upload a large bag or anything containing private camera/audio data without review. Write measured results only after you run the experiment.

What failure testing should reveal

A browser can request zero speed when the operator releases a key, but it cannot guarantee delivery after a link or process failure. Inspect the Gazebo diff-drive command timeout, if any, and measure it rather than assuming one exists. A stronger independent design places a timeout near the actuator or simulation controller and publishes or applies zero speed when commands become stale. Record the time from last valid command to stop under repeated disconnects. This is the main engineering distinction between a demo interface and a dependable teleoperation path.

Original follow-on project brief

Build a separate Teleoperation Reliability Logger after reproducing the baseline. Keep the upstream project as a cited reference. Your project can use a fresh ROS 2 node that accepts operator Twist commands, stamps them, checks age and speed limits, forwards safe commands, and records command, odometry and fault events. Add a small test harness that injects delayed commands and link loss. Do not present this design as completed until code and measurements exist.

Deliverable	Minimum acceptance check
Robot-side timeout	A stale command leads to zero within a documented target time in repeated tests
Command limits	Linear and angular outputs never exceed configured caps
Episode log	Bag or structured log contains command, state, stop reason and timestamps
Test evidence	At least normal drive, release, disconnect and delayed-message cases
Project report	Architecture, actual plots, failure table, reproducible instructions and limits

A candidate target is a 250 ms command-age limit, but choose the final value from controller frequency, network measurements and safe stopping behavior. The number is a design goal, not a claimed result.

Four week learning plan

Week	Output you should personally produce
1	ROS 2 and Gazebo installation notes, package build, topic and frame map
2	Recorded baseline drive, odometry and scan; explanation of every path in Figure
3	Failure experiments and a written timeout design with test cases
4	A small independent node or simulator extension, tests and measured report

Interview questions to practise

- What is the difference between /cmd_vel and /odom, and why can they disagree?
- What does ros_gz_bridge do, and how is it different from rosbridge?
- What happens if the browser disappears after sending a nonzero command?
- How would you measure the delay from operator action to robot motion?
- What would you record in a teleoperation episode for debugging and later learning?

What can honestly appear on a resume

After a verified run, describe the specific package you built, topics you inspected, faults you reproduced and code you personally changed. Until then, list this only as a learning reference. A strong original entry includes measured latency or stop time, a test method and a link to your own repository.

Sources and attribution

Pinned upstream project https://github.com/weepwood/RoboPilot

Upstream Windows setup https://github.com/weepwood/RoboPilot/blob/main/docs/WINDOWS_WSL2.md

Gazebo and ROS pairing https://gazebosim.org/docs/jetty/ros_installation/

ROS 2 Jazzy documentation https://docs.ros.org/en/jazzy/

rosbag2 recording and playback https://github.com/ros2/rosbag2